

LAB10_WebClient

1. Reactive และ Blocking

Blocking I/O ทำให้ thread รอผลของงาน I/O ก่อนดำเนินงานต่อ ขณะที่ non-blocking I/O

ลงทะเบียนรับผลแล้วคืนการควบคุมให้ runtime จัดการงานอื่นได้ Spring WebFlux และ Reactor Netty ใช้ event loop ในการรับส่ง HTTP ส่วน WebClient คืน Publisher ของผลลัพธ์ที่ยังไม่จำเป็นต้องเสร็จทันที

Reactive ไม่ได้ทำให้ทุกคำสั่งเป็น asynchronous โดยอัตโนมัติ การคำนวณราคาและการเข้าถึง Map ยังเป็นงานสั้น ๆ บน CPU ของ thread ที่ประมวลผล ส่วนการรอ HTTP ของ WebClient ไม่บล็อก thread ในโค้ด endpoint ไม่มี block(), sleep(), Future.get(), JDBC หรือ file I/O

Mono.defer และ Flux.defer เลื่อนการสร้าง stream จน subscribe; Mono.fromSupplier และ

Mono.fromRunnable เลื่อนการอ่านหรือแก้ไขข้อมูลในหน่วยความจำ วิธีนี้ไม่ได้ย้ายงานไปอีก thread และไม่ควรรนำ I/O แบบ blocking มาใส่แล้วถือว่าเป็น non-blocking

2. Mono, Flux

ชนิด	จำนวนข้อมูล	ใช้กรณีไหน	เพราะอะไร
Mono<T>	0 หรือ 1 ค่า	ค้นหาสินค้าด้วย ID, บันทึกสินค้า, ดูราคาหลังลด	แต่ละงานต้องการผลลัพธ์เพียงค่าเดียว
Flux<T>	0 ถึงหลายค่า	ดูสินค้าทั้งหมด, ค้นหาตาม category	ผลลัพธ์อาจมีสินค้าหลายรายการ
Mono<Void>	ไม่มีข้อมูล ส่งเพียงสัญญาณสำเร็จหรือข้อผิดพลาด	ลบสินค้า	ต้องรู้ว่าจะงานเสร็จหรือไม่ โดยไม่ต้องคืนข้อมูลสินค้า

673380392-1 นายกานดิทัต นามสุดตา Sec 3

3. WebClient method chain

get(), post(), delete() เลือก HTTP method; uri() กำหนด path และ path variables; bodyValue() ใส่ request body; retrieve() กำหนดการดึง response และจัดการ error status; bodyToMono() หรือ bodyToFlux() กำหนดการแปลง response body เป็นชนิดข้อมูลที่ต้องการ การส่ง request จริงเกิดเมื่อ subscribe

4. โค้ด

โค้ด: ProductRepository.java

```
package com.example.lab10.repository;
import com.example.lab10.model.Product;
import reactor.core.publisher.Flux;
import reactor.core.publisher.Mono;
import java.util.Map;
import java.util.concurrent.ConcurrentHashMap;

public class ProductRepository {

    // — In-memory storage —
    private final Map<String, Product> store = new ConcurrentHashMap<>();

    // — Constructor: ใส่ข้อมูลตัวอย่าง —
    public ProductRepository() {
        store.put("1", new Product("1", "iPhone 15 Pro (นายกานดิทัต นามสุดตา 673380392-1 SEC 3)",
            "Electronics", "Apple", 50, 39900.0, "MEMBER"));
        store.put("2", new Product("2", "MacBook Air M3",
            "Electronics", "Apple", 20, 49900.0, "NONE"));
        store.put("3", new Product("3", "Samsung Galaxy S24",
            "Electronics", "Samsung", 30, 29900.0, "SEASONAL"));
    }

    // — 1. หา Product 1 รายการ —
    // Read at subscription time; missing key completes without a value.
    public Mono<Product> findById(String id) {
        return Mono.defer(() -> Mono.justOrEmpty(store.get(id)));
    }

    // — 2. หา Product ทั้งหมด —
    // Publish current in-memory products lazily.
    public Flux<Product> findAll() {
        return Flux.defer(() -> Flux.fromIterable(store.values()));
    }

    // — 3. บันทึก Product —
```

```
// Save only after the publisher is subscribed.
public Mono<Product> save(Product product) {
    return Mono.fromSupplier(() -> {
        store.put(product.getId(), product);
        return product;
    });
}

// — 4. ลบ Product —
// Completion-only publisher for the deferred side effect.
public Mono<Void> deleteById(String id) {
    return Mono.fromRunnable(() -> store.remove(id));
}

// — 5. กรองตาม category —
// filter keeps matching elements without collecting or blocking.
public Flux<Product> findByCategory(String category) {
    return findAll().filter(p -> category.equalsIgnoreCase(p.getCategory()));
}
}
```

โค้ด: ProductService.java

```
package com.example.lab10.service;
import com.example.lab10.model.Product;
import com.example.lab10.repository.ProductRepository;
import org.springframework.stereotype.Service;
import reactor.core.publisher.Flux;
import reactor.core.publisher.Mono;
@Service
public class ProductService {
    // — Constructor Injection (DIP — SOLID) —
    private final ProductRepository repository;
    public ProductService(ProductRepository repository) {
        this.repository = repository;
    }

    // — 1. ดึง Product 1 รายการ —
    public Mono<Product> getById(String id) {
        return repository.findById(id).switchIfEmpty(Mono.error(
            new org.springframework.web.server.ResponseStatusException(
                org.springframework.http.HttpStatus.NOT_FOUND, "Product not found: " + id)));
    }
}
```

```

}
// — 2. ดึง Product ทั้งหมด —
public Flux<Product> getAll() {
    return repository.findAll();
}

// — 3. บันทึก Product —
// Save only after the publisher is subscribed.
public Mono<Product> save(Product product) {
    // Defer validation, ID generation and write until subscription.
    return Mono.defer(() -> {
        if (product == null || product.getName() == null || product.getName().isBlank()
            || product.getCategory() == null || product.getCategory().isBlank()
            || product.getPrice() == null || !Double.isFinite(product.getPrice())
            || product.getPrice() < 0 || product.getStock() == null || product.getStock() <
0
            || (product.getDiscountType() != null && !java.util.Set.of(
                "NONE", "MEMBER", "SEASONAL").contains(product.getDiscountType())) {
            return Mono.error(new org.springframework.web.server.ResponseStatusException(
                org.springframework.http.HttpStatus.BAD_REQUEST, "Invalid product fields"));
        }
        if (product.getId() == null || product.getId().isBlank())
            product.setId(java.util.UUID.randomUUID().toString());
        if (product.getDiscountType() == null) product.setDiscountType("NONE");
        return repository.save(product);
    });
}

// — 4. ลบ Product —
public Mono<Void> delete(String id) {
    return getByid(id).flatMap(p -> repository.deleteByid(p.getId()));
}

// — 5. กรองตาม category —
public Flux<Product> getByCategory(String category) {
    return repository.findByCategory(category);
}

// — 6. คำนวณราคาหลังส่วนลด —
// A single numeric result; callers can continue chaining operators.
public Mono<Double> getDiscountedPrice(String id) {
    return getByid(id).map(Product::getDiscountedPrice);
}

```

```

}
}

```

โค้ด: ProductController.java

```

package com.example.lab10.controller;
import com.example.lab10.model.Product;
import com.example.lab10.service.ProductService;
import org.springframework.web.bind.annotation.*;
import reactor.core.publisher.Flux;
import reactor.core.publisher.Mono;
@RestController
@RequestMapping("/products")
public class ProductController {
    // — Constructor Injection (DIP — SOLID) —
    private final ProductService service;
    public ProductController(ProductService service) {
        this.service = service;
    }
    @GetMapping("/{id}")
    public Mono<Product> getById(@PathVariable String id) {
        return service.getById(id);
    }
    @GetMapping
    public Flux<Product> getAll() {
        return service.getAll();
    }
    @PostMapping
    public Mono<Product> save(@RequestBody Product product) {
        return service.save(product);
    }
    @DeleteMapping("/{id}")
    @ResponseStatus(org.springframework.http.HttpStatus.NO_CONTENT)
    public Mono<Void> delete(@PathVariable String id) {
        return service.delete(id);
    }
    @GetMapping("/category/{category}")
    public Flux<Product> getByCategory(@PathVariable String category) {
        return service.getByCategory(category);
    }
}

```

```

    }
    @GetMapping("/{id}/price")
    // A single numeric result; callers can continue chaining operators.
    public Mono<Double> getDiscountedPrice(@PathVariable String id) {
        return service.getDiscountedPrice(id);
    }
}

```

โค้ด: ProductWebClient.java

```

package com.example.lab10.client;
import com.example.lab10.model.Product;
import org.springframework.stereotype.Component;
import org.springframework.web.reactive.function.client.WebClient;
import reactor.core.publisher.Flux;
import reactor.core.publisher.Mono;
@Component
public class ProductWebClient {
    // HTTP client targeting this application by default.
    private final WebClient client;

    public ProductWebClient(@org.springframework.beans.factory.annotation.Value(
        "${product.api.base-url:http://localhost:${server.port:8080}}") String baseUrl) {
        this.client = WebClient.builder().baseUrl(baseUrl).build();
    }

    // HTTP 404 is an error, not an empty Mono. Convert only 404 before defaultIfEmpty.
    public Mono<String> getNameOrDefault(String id) {
        return getProductById(id).map(Product::getName)
            .onErrorResume(org.springframework.web.reactive.function.client.WebClientResponseExcepti
on.NotFound.class,
                error -> Mono.empty())
            .defaultIfEmpty("Product not found");
    }

    public Mono<Product> getProductById(String id) {
        return client.get()
            .uri("/products/{id}", id)
            .retrieve()
            .bodyToMono(Product.class);
    }

    public Flux<Product> getAllProducts() {

```

673380392-1 นายกานดิทัต นามสุดตา Sec 3

```
        return client.get().uri("/products").retrieve().bodyToFlux(Product.class);
    }

    public Mono<Product> createProduct(Product product) {
        return client.post().uri("/products").bodyValue(product)
            .retrieve().bodyToMono(Product.class);
    }

    public Mono<Void> deleteProduct(String id) {
        return client.delete().uri("/products/{id}", id).retrieve().bodyToMono(Void.class);
    }

    public Flux<Product> getByCategory(String category) {
        return client.get().uri("/products/category/{category}", category)
            .retrieve().bodyToFlux(Product.class);
    }

    // A single numeric result; callers can continue chaining operators.
    public Mono<Double> getDiscountedPrice(String id) {
        return client.get().uri("/products/{id}/price", id).retrieve()
            .bodyToMono(Double.class); // Caller may chain operators without blocking.
    }
}
```

673380392-1 นายกานดิทัต นามสุดตา Sec 3

โค้ด: ClientDemoController.java

```
package com.example.lab10.controller;

import com.example.lab10.client.ProductWebClient;
import org.springframework.web.bind.annotation.*;
import reactor.core.publisher.Mono;

@RestController
@RequestMapping("/client-demo")
public class ClientDemoController {
    private final ProductWebClient client;

    public ClientDemoController(ProductWebClient client) { this.client = client; }

    // Spring subscribes to this returned publisher; no manual subscribe in controllers.
    @GetMapping("/products/{id}/name")
    public Mono<String> name(@PathVariable String id) { return client.getNameOrDefault(id); }
}
```

โค้ด: WalletService.java

```
package com.example.lab10.service;
import java.math.BigDecimal;
import java.math.RoundingMode;
import java.util.concurrent.atomic.AtomicReference;
import org.springframework.stereotype.Service;
import org.springframework.http.HttpStatus;
import org.springframework.web.server.ResponseStatusException;
import reactor.core.publisher.Mono;
@Service
public class WalletService {
    private final AtomicReference<BigDecimal> balance = new AtomicReference<> (new
    BigDecimal("0.00"));
    public Mono<BigDecimal> balance() { return Mono.fromSupplier(balance::get); }
    public Mono<BigDecimal> deposit(BigDecimal amount) {
        return Mono.defer(() -> {
            if (amount == null || amount.signum() <= 0)
                return Mono.error(new ResponseStatusException(HttpStatus.BAD_REQUEST, "Amount must
                be positive"));
            final BigDecimal normalized;
            try { normalized = amount.setScale(2, RoundingMode.UNNECESSARY); }
            catch (ArithmeticException ex) {
                return Mono.error(new ResponseStatusException(HttpStatus.BAD_REQUEST, "At most 2
                decimal places"));
            }
            return Mono.fromSupplier(() -> balance.updateAndGet(old -> old.add(normalized)));
        });
    }
}
```

โค้ด: WalletController.java

```
package com.example.lab10.controller;
import java.math.BigDecimal;
import com.example.lab10.service.WalletService;
import org.springframework.web.bind.annotation.*;
import reactor.core.publisher.Mono;
@RestController
@RequestMapping("/wallet")
public class WalletController {
    public record Deposit(BigDecimal amount) {}
    public record Balance(BigDecimal balance) {}
    private final WalletService service;
    public WalletController(WalletService service) { this.service = service; }
    @GetMapping
    public Mono<Balance> balance() { return service.balance().map(Balance::new); }
    @PostMapping("/deposits")
    public Mono<Balance> deposit(@RequestBody Mono<Deposit> request) {
        return request.flatMap(d -> service.deposit(d.amount())).map(Balance::new);
    }
}
```

5. ผลลัพธ์

GET <http://localhost:8080/products/1>

id:	"1"
name:	"iPhone 15 Pro (นายกานดิทัต นามสุดตา 673380392-1 SEC 3)"
category:	"Electronics"
brand:	"Apple"
stock:	50
price:	39900.0 JS: 39900
discountType:	"MEMBER"
discountedPrice:	35910.0 JS: 35910

GET <http://localhost:8080/products>

▼ 0:	
id:	"1"
name:	"iPhone 15 Pro (นายกานดิทัต นามสุดตา 673380392-1 SEC 3)"
category:	"Electronics"
brand:	"Apple"
stock:	50
price:	39900.0 JS: 39900
discountType:	"MEMBER"
discountedPrice:	35910.0 JS: 35910
▼ 1:	
id:	"2"
name:	"MacBook Air M3"
category:	"Electronics"
brand:	"Apple"
stock:	20
price:	49900.0 JS: 49900
discountType:	"NONE"
discountedPrice:	49900.0 JS: 49900
▼ 2:	
id:	"3"
name:	"Samsung Galaxy S24"
category:	"Electronics"
brand:	"Samsung"
stock:	30
price:	29900.0 JS: 29900
discountType:	"SEASONAL"
discountedPrice:	23920.0 JS: 23920

673380392-1 นายกานดิทัต นามสุดตา Sec 3

POST http://localhost:8080/products

The screenshot displays a REST client interface with the following details:

- URL:** http://localhost:8080/products
- Method:** POST
- Body Type:** raw (selected)
- Request Body (raw):**

```
1 {  
2   "id": "4",  
3   "name": "Keyboard (นายกานดิทัต นามสุดตา 673380392-1 SEC 3)",  
4   "category": "Accessories",  
5   "brand": "Logitech",  
6   "stock": 10,  
7   "price": 1000.0,  
8   "discountType": "MEMBER"  
9 }
```
- Response Status:** 200
- Response Body (JSON):**

```
1 {  
2   "id": "4",  
3   "name": "Keyboard (นายกานดิทัต นามสุดตา 673380392-1 SEC 3)",  
4   "category": "Accessories",  
5   "brand": "Logitech",  
6   "stock": 10,  
7   "price": 1000.0,  
8   "discountType": "MEMBER",  
9   "discountedPrice": 900.0  
10 }
```

673380392-1 นายกานดิท นามสุดตา Sec 3

GET http://localhost:8080/products/category/Electronics

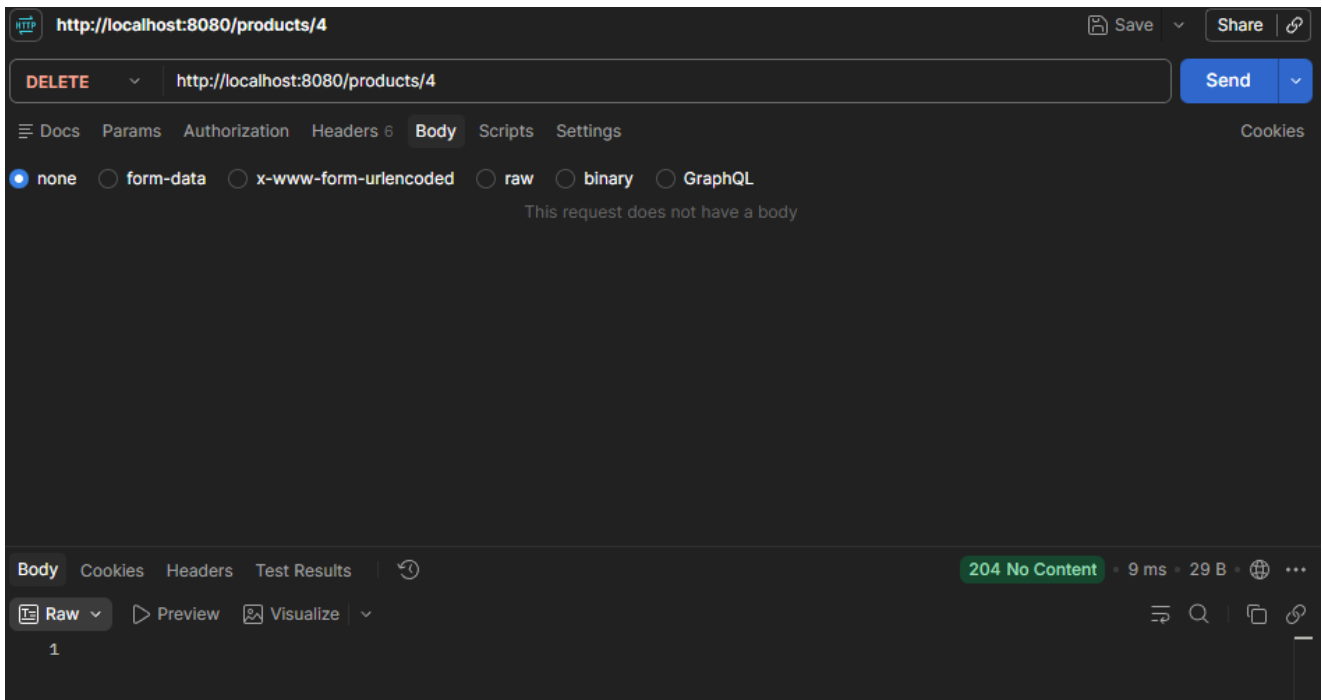
▼ 0:	
id:	"1"
name:	"iPhone 15 Pro (นายกานดิท นามสุดตา 673380392-1 SEC 3)"
category:	"Electronics"
brand:	"Apple"
stock:	50
price:	39900.0 JS: 39900
discountType:	"MEMBER"
discountedPrice:	35910.0 JS: 35910
▼ 1:	
id:	"2"
name:	"MacBook Air M3"
category:	"Electronics"
brand:	"Apple"
stock:	20
price:	49900.0 JS: 49900
discountType:	"NONE"
discountedPrice:	49900.0 JS: 49900
▼ 2:	
id:	"3"
name:	"Samsung Galaxy S24"
category:	"Electronics"
brand:	"Samsung"
stock:	30
price:	29900.0 JS: 29900
discountType:	"SEASONAL"
discountedPrice:	23920.0 JS: 23920

GET http://localhost:8080/products/1/price

35910.0 JS: 35910

673380392-1 นายกานดิท นามสุดา Sec 3

DELETE http://localhost:8080/products/4



673380392-1 นายกานดิท นามสุดา Sec 3

GET http://localhost:8080/client-demo/products/1/name

iPhone 15 Pro (นายกานดิท นามสุดา 673380392-1 SEC 3)

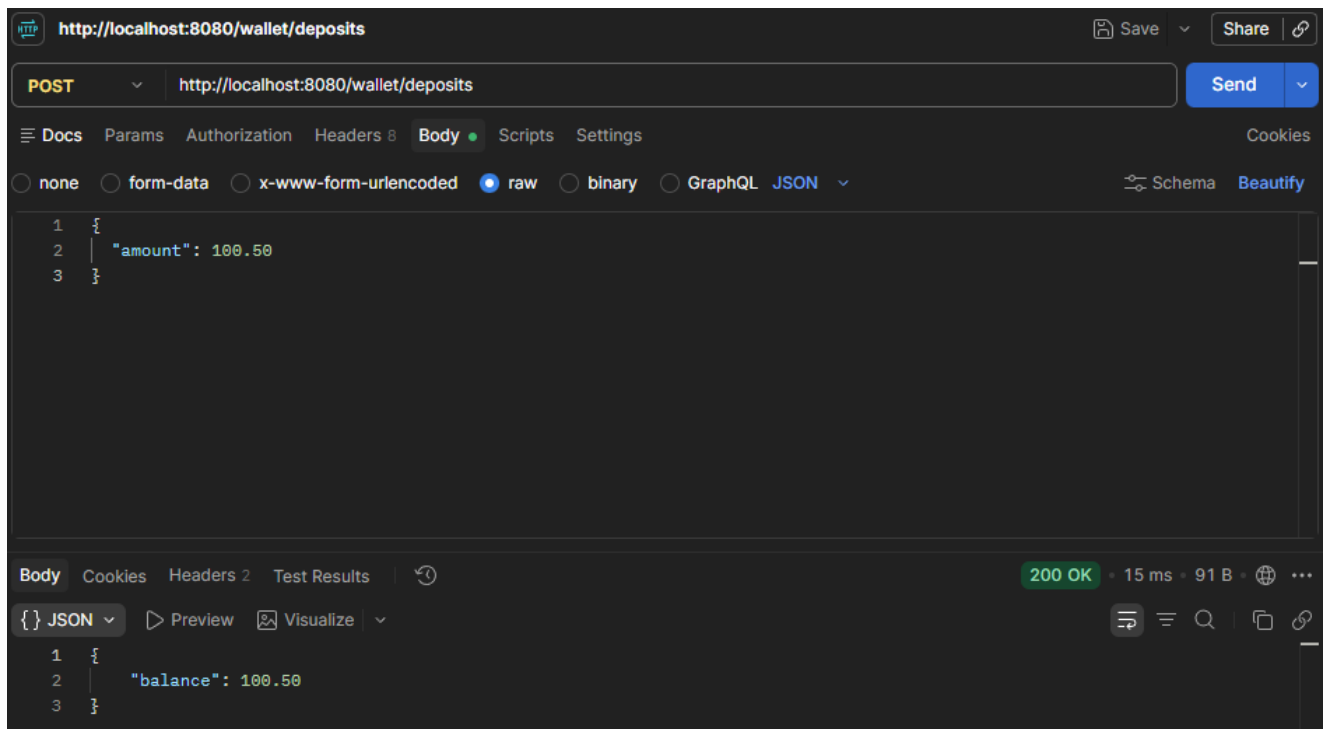
GET http://localhost:8080/client-demo/products/missing/name

Product not found

GET http://localhost:8080/wallet

```
balance: 0.00 JS: 0
```

POST http://localhost:8080/wallet/deposits



```
balance: 100.50 JS: 100.5
```